



ELTE  
FACULTY OF  
INFORMATICS

# PROGRAMMING

## Lecture 5

Zsuzsa Pluhár  
pluharzs@inf.elte.hu

# Refreshing



# Programming patterns

---

1. Summation
2. Counting
3. Maximum selection
4. Copy
5. Multiple item selection
6. Search
7. Decision
8. Selection

*problems with sum and "all"*  
↓  
*counting loop*

*problem with "exists"*  
↓  
*conditional loop*



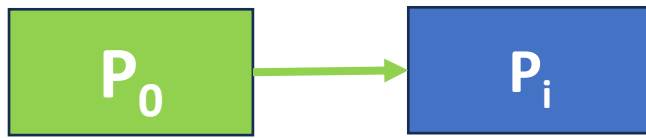
# More programming patterns



# Using more patterns

Complex problems – we need more patterns

- pattern as a function -> combine them



$P_0$ : output is a sequence  $\Rightarrow$  copy, multiple item selection, sorting, ...

$P_i$ : all patterns



$P_0$ : output is a boolean value (a single value)  
 $\Rightarrow$  decision, search, ...

$P_i$ : patterns contain an „attribute function”

More programming patterns  
one after the other



# Example – books

---

A library store the borrowings. They would like to know whether proportionally more girls read Terry Pratchett's books than boys read Neil Gaiman's.



# Example – books



A library store the borrowings. They would like to know whether proportionally more girls read Terry Pratchett's books than boys read Neil Gaiman's.

| Book        | Sex |
|-------------|-----|
| T.Pratchett | G   |
| N.Gaiman    | G   |
| N.Gaiman    | B   |
| T.Pratchett | G   |
| T.Pratchett | B   |
| T.Pratchett | G   |
| N.Gaiman    | G   |
| T.Pratchett | G   |
| N.Gaiman    | B   |
| N.Gaiman    | B   |
| T.Pratchett | G   |
| T.Pratchett | G   |
| N.Gaiman    | B   |

→ T.Pratchett 7  
→ N.Gaiman 6  
→ T.Pratchett and girl 6  
→ N.Gaiman and boy 4

→ T.Pratchett: 6/7  
→ N.Gaiman: 4/6

→ true

4 „counting”s



# Example – books

A library store the borrowings. They would like to know whether proportionally more girls read Terry Pratchett's books than boys read Neil Gaiman's.

## Specification:

In:  $n \in \mathbb{N}$ ,  $\text{readers} \in \text{Tr}[1..n]$ ,  $\text{Tr} = (\text{book}:S \times \text{sex}:C)$

Out:  $\text{gaimanb} \in \mathbb{L}$

Aux:  $\text{cnttp} \in \mathbb{N}$ ,  $\text{cntng} \in \mathbb{N}$ ,  $\text{cntg} \in \mathbb{N}$ ,  $\text{cntb} \in \mathbb{N}$

Pre:  $\forall i \in [1..n]: (\text{readers}[i].\text{sex} = "G" \text{ or } \text{readers}[i].\text{sex} = "B")$

Post:

$\text{cnttp} = \text{COUNT}(i=1..n, \text{readers}[i].\text{book} = "T.Pratchett")$  and  
 $\text{cntng} = \text{COUNT}(i=1..n, \text{readers}[i].\text{book} = "N.Gaiman")$  and  
 $\text{cntg} = \text{COUNT}(i=1..n, \text{readers}[i].\text{book} = "T.Pratchett" \text{ and } \text{readers}[i].\text{sex} = "G")$  and  
 $\text{cntb} = \text{COUNT}(i=1..n, \text{readers}[i].\text{book} = "N.Gaiman" \text{ and } \text{readers}[i].\text{sex} = "B")$  and  
 $\text{gaimanb} = (\text{cng} / \text{cnttp}) > (\text{cntb} / \text{cntng})$

# Example – books



Specification: pattern: `cnt=COUNT(i=b..e, A(i))`

Post:

```
cnttp = COUNT(i=1..n, readers[i].book="T.Pratchett") and  
cntng = COUNT(i=1..n, readers[i].book="N.Gaiman") and  
cntg  = COUNT(i=1..n, readers[i].book="T.Pratchett" and  
        readers[i].sex="G") and  
cntb  = COUNT(i=1..n, readers[i].book="N.Gaiman" and  
        readers[i].sex="B") and
```

```
cnt    ~ cnttp  
b..e   ~ 1..n  
A(i)   ~ readers[i].book="T.Pratchett"
```

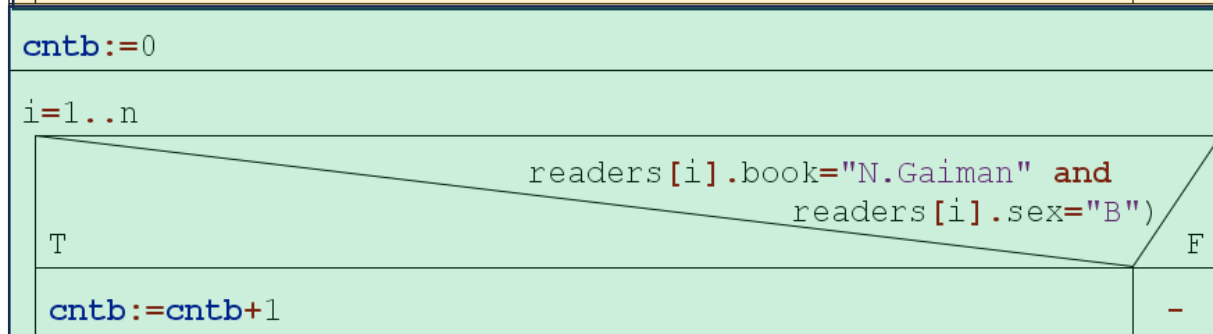
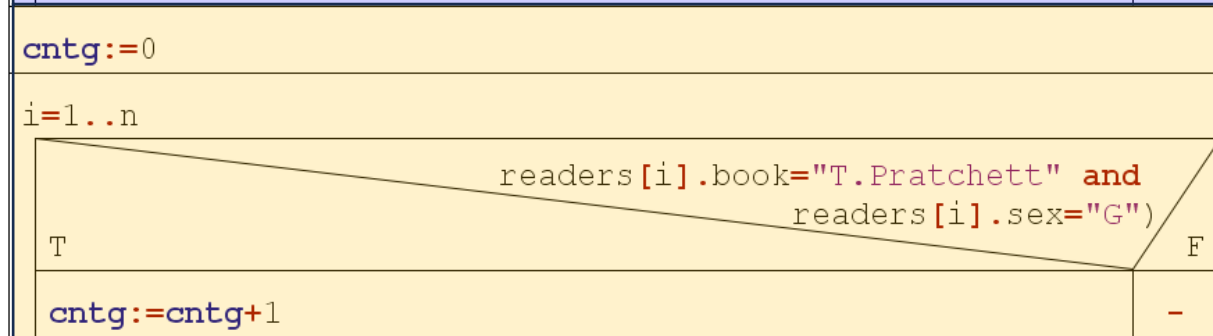
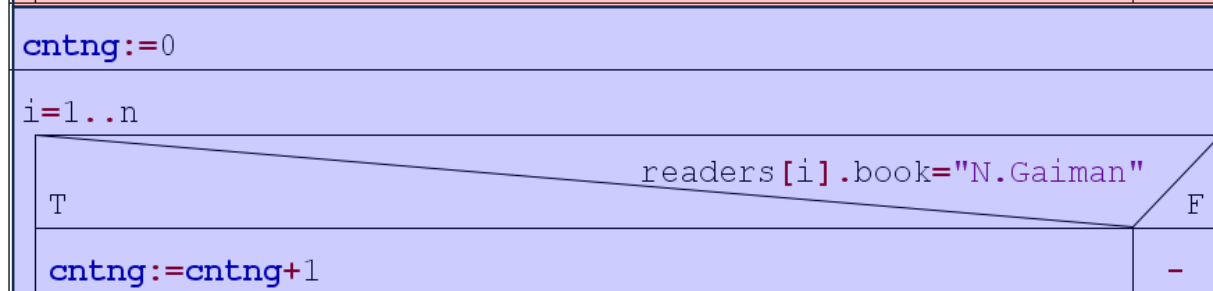
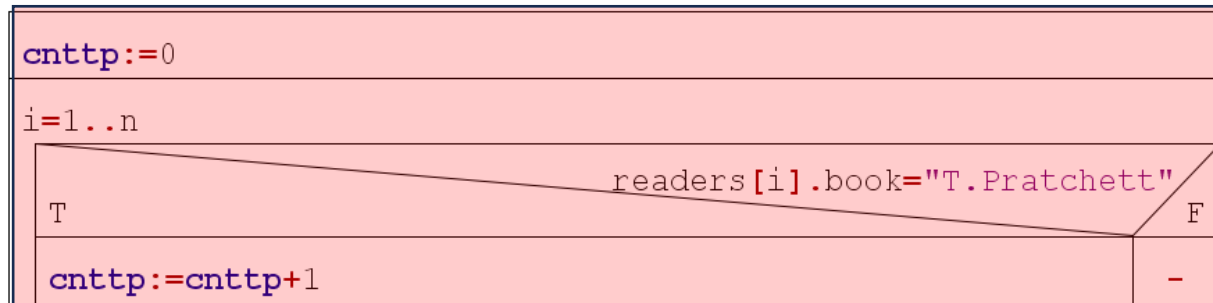
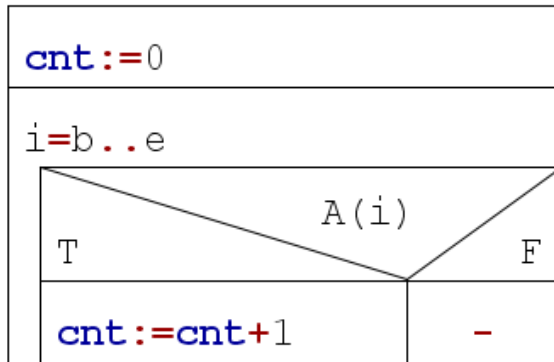
```
cnt    ~ cntg  
b..e   ~ 1..n  
A(i)   ~ readers[i].book="T.Pratchett" and readers[i].sex="G"
```

```
cnt    ~ cntng  
b..e   ~ 1..n  
A(i)   ~ readers[i].book="N.Gaiman"
```

```
cnt    ~ cntb  
b..e   ~ 1..n  
A(i)   ~ readers[i].book="N.Gaiman" and readers[i].sex="B"
```



# Example – k



`gaimanb := ((cntg / cnttp) > (cntb / cntng))`

`cnt` ~ `cnttp`  
`b..e` ~ `1..n`  
`A(i)` ~ `readers[i].book = "T.Pratchett"`

`cnt` ~ `cntng`  
`b..e` ~ `1..n`  
`A(i)` ~ `readers[i].book = "N.Gaiman"`

`cnt` ~ `cntg`  
`b..e` ~ `1..n`  
`A(i)` ~ `readers[i].book = "T.Pratchett" and readers[i].sex = "G"`

`cnt` ~ `cntb`  
`b..e` ~ `1..n`  
`A(i)` ~ `readers[i].book = "N.Gaiman" and readers[i].sex = "B"`

# Example – books



## Conclusions:

- in specification one after the other
- in algorithm too – sequence of „instructions”
- helper/auxiliary variable

i  
n  
p  
u  
t

| Book        | Sex |
|-------------|-----|
| T.Pratchett | G   |
| N.Gaiman    | G   |
| N.Gaiman    | B   |
| T.Pratchett | G   |
| T.Pratchett | B   |
| T.Pratchett | G   |
| N.Gaiman    | G   |
| T.Pratchett | G   |
| N.Gaiman    | B   |
| N.Gaiman    | B   |
| T.Pratchett | G   |
| T.Pratchett | G   |
| N.Gaiman    | B   |

→ T.Pratchett 7  
→ N.Gaiman 6  
→ T.Pratchett and girl 6  
→ N.Gaiman and boy 4

helper variable

→ T.Pratchett: 6/7  
→ N.Gaiman: 4/6

→ true

o  
u  
t  
p  
u  
t

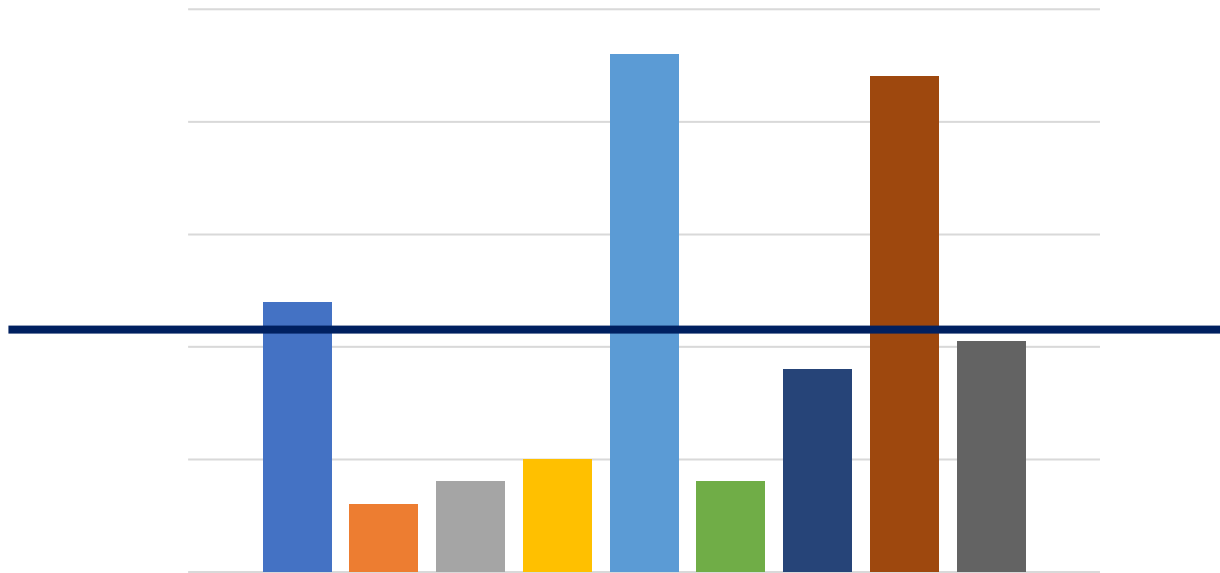


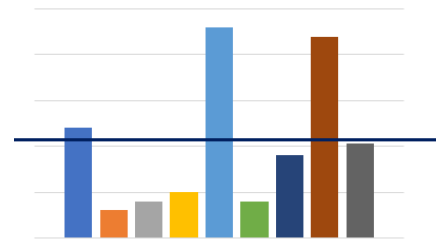
E

# Example – grades

---

We store the grades of  $N$  students. Let's list the students over the average grade.





# Example – grades

We store the grades of N students. Let's list the students over the average grade.

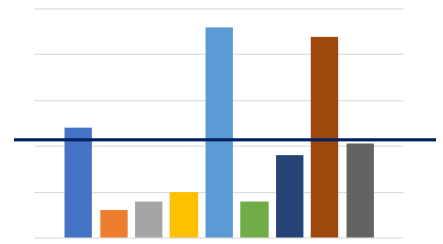
| grades |
|--------|
| 12     |
| 3      |
| 4      |
| 5      |
| 23     |
| 4      |
| 9      |
| 22     |

→ average: 10.25

summation

| grades | students |
|--------|----------|
| 12     | 1        |
| 3      | 5        |
| 4      | 8        |
| 5      |          |
| 23     |          |
| 4      |          |
| 9      |          |
| 22     |          |

multiple item selection



# Example – grades

We store the grades of  $N$  students. Let's list the students over the average grade.

## Specification:

In:  $n \in \mathbb{N}$ ,  $\text{grades} \in \mathbb{N}[1..n]$

Out:  $\text{cnt} \in \mathbb{N}$ ,  $\text{students} \in \mathbb{N}[1..\text{cnt}]$

Aux:  $\text{av} \in \mathbb{R}$

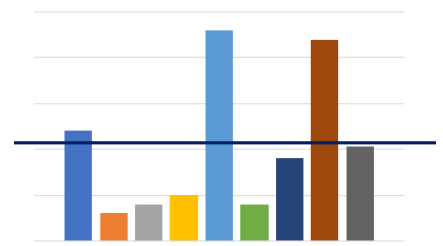
Pre:  $\forall i \in [1..n]: (1 \leq \text{grades}[i] \leq 5)$

Post:  $\text{av} = \text{SUM}(i=1..n, \text{grades}[i]) / n$  and

$(\text{cnt}, \text{students}) = \text{MIS}(i=1..n, \text{grades}[i] > \text{av}, i)$

$s = \text{SUM}(i=b..e, f(i))$

$(\text{cnt}, y) = \text{MIS}(i=b..e, A(i), f(i))$



# Example – grades

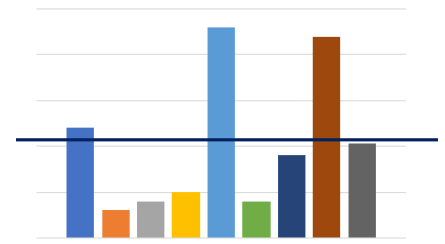
Post:  $av = \text{SUM}(i=1..n, \text{grades}[i]) / n$  and

$s = \text{SUM}(i=b..e, f(i))$

|        |        |                    |
|--------|--------|--------------------|
| $s$    | $\sim$ | $av * n$           |
| $b..e$ | $\sim$ | $1..n$             |
| $A(i)$ | $\sim$ | $\text{grades}[i]$ |

|                 |
|-----------------|
| $s := 0$        |
| $i = b..e$      |
| $s := s + f(i)$ |

|                             |
|-----------------------------|
| $s := 0$                    |
| $i = 1..n$                  |
| $s := s + \text{grades}[i]$ |
| $av := s / n$               |

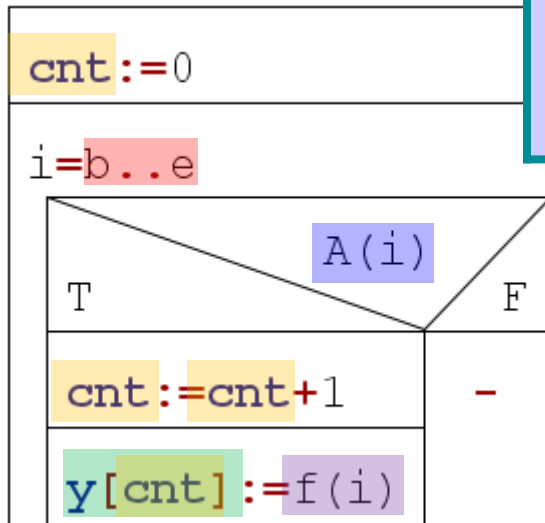


# Example – grades

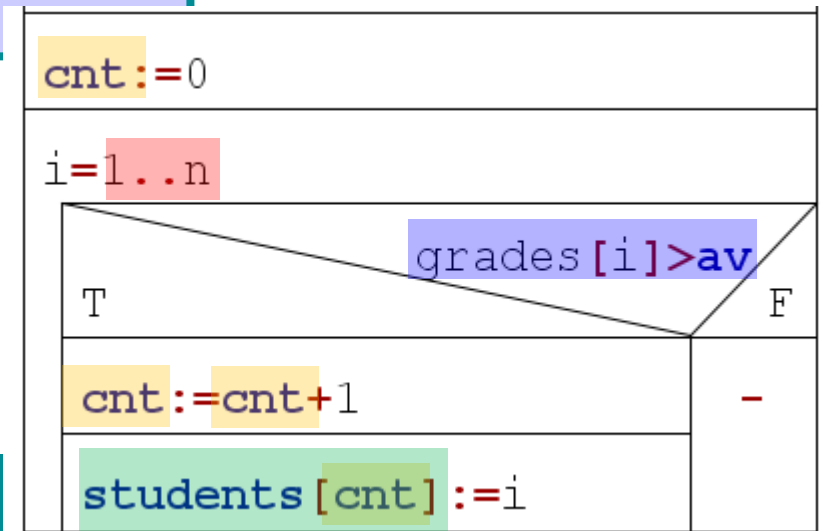
Post: ... and

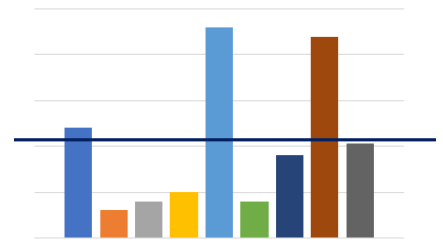
$(cnt, students) = MIS(i=1..n, grades[i] > av, i)$

$(cnt, y) = MIS(i=b..e, A(i), f(i))$



|        |        |                  |
|--------|--------|------------------|
| $cnt$  | $\sim$ | $cnt$            |
| $b..e$ | $\sim$ | $1..n$           |
| $A(i)$ | $\sim$ | $grades[i] > av$ |
| $f(i)$ | $\sim$ | $i$              |

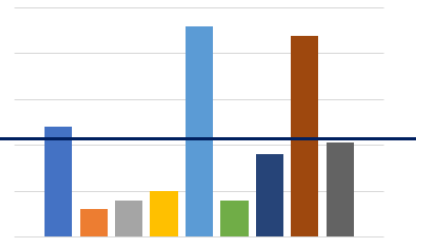




# Example – grades

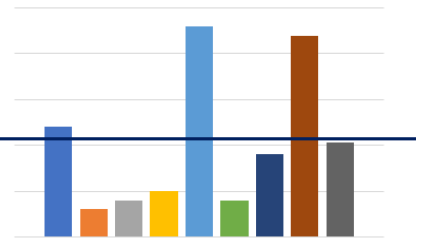
|                                                                     |   |
|---------------------------------------------------------------------|---|
| <code>s := 0</code>                                                 |   |
| <code>i = 1..n</code>                                               |   |
| <code>s := s + grades[i]</code>                                     |   |
| <code>av := s / n</code>                                            |   |
| <code>cnt := 0</code>                                               |   |
| <code>i = 1..n</code>                                               |   |
| <div> <div>grades[i] &gt; av</div> <div>T</div> <div>F</div> </div> |   |
| <code>cnt := cnt + 1</code>                                         | - |
| <code>students[cnt] := i</code>                                     |   |

# Example – grades, the code



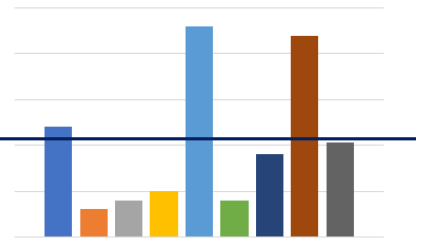
```
static void Main(string[] args) {  
    // declaring (variables, specification: in/out)  
    int n;  
    int[] grades;  
    double av;  
    int cnt;  
    List<int> students = new List<int>();  
  
    // Reading the input  
    readIn(out n, out grades);  
    av=average(n,grades);  
    (cnt,students)=mis_students(n,grades,av);  
    writeOut(cnt,students);  
}
```

# Example – grades, the code



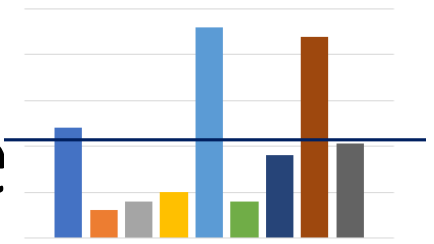
```
static void Main(string[] args) {  
  
    int n;  
    int[] grades;  
    double av;  
    int cnt;  
    List<int> students = new List<int>();  
  
    readIn(out grades);  
    av=average(grades);  
    students=mis_students(grades,av);  
    writeOut(students);  
}
```

# Example – grades, the code



```
static void readIn(out int[] x) {
    int n;
    Console.Error.Write("n = ");
    int.TryParse(Console.ReadLine(), out n);
    x = new int[n];
    for (int i = 0; i < n; i++) {
        Console.Error.Write("{0}. grade = ", i + 1);
        int.TryParse(Console.ReadLine(), out x[i]);
    }
}

static void writeOut(List<int> x) {
    for (int i = 0; i < x.Count; i++) {
        Console.Error.Write("{0}. student; ", x[i] + 1);
    }
}
```

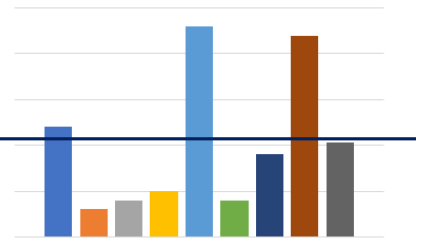


# Example – grades, the code

```
static double average(int[] x) {
    int sum = 0;
    for (int i=0;i<x.Length;i++) {
        sum += x[i];
    }
    double sav = (double)sum / x.Length;
    return sav;
}

static List<int> mis_students(int[] x,double y) {
    List<int> st = new List<int>();
    for (int i= 0; i < x.Length; i++) {
        if (x[i]>y) {
            st.Add(i);
        }
    }
    return st;
}
```

# Example – grades, the code



```
static void Main(string[] args) {  
    int[] grades;  
    List<int> students = new List<int>();  
  
    readIn(out grades);  
    students = mis_students(grades, average(grades));  
    writeOut(students);  
}
```



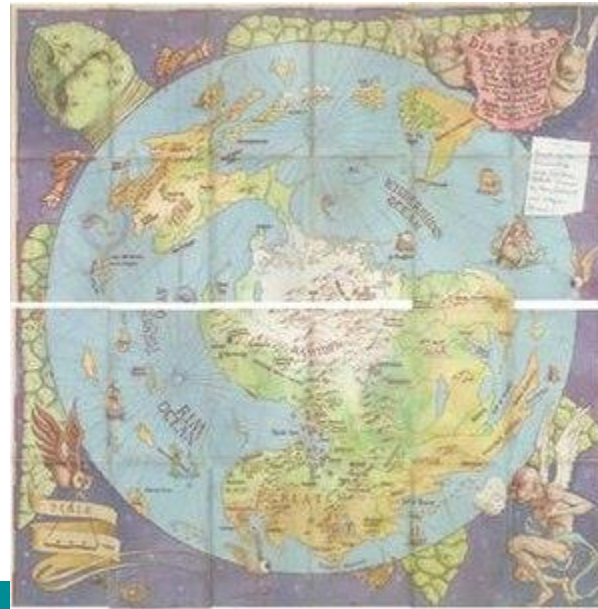
More programming patterns  
nested in each other



# Example - intersection



The Librarian stores the locations that were visited by the Chair of the Indefinite Studies and the Lecturer in Recent Runes. He wants to make a list of places where both wizards have been?



# Example - intersection



**Task:** We have two sequences and would like to **list all items** that **can be found in both**.

| C.I.S.       |
|--------------|
| Ankh-Morpork |
| Klatch       |
| Überwald     |
| Pseudopolis  |

| L.R.R.       |
|--------------|
| Holy Wood    |
| Ankh-Morpork |
| Quirm        |
| Lancre       |
| Überwald     |



| both         |
|--------------|
| Ankh-Morpork |
| Überwald     |

# Example - intersection



**Task:** We have two sequences and would like to **list all items** that **can be found in both**.

| C.I.S.       |
|--------------|
| Ankh-Morpork |
| Klatch       |
| Überwald     |
| Pseudopolis  |

| L.R.R.       |
|--------------|
| Holy Wood    |
| Ankh-Morpork |
| Quirm        |
| Lancre       |
| Überwald     |



| both         |
|--------------|
| Ankh-Morpork |
| Überwald     |

List all item with a given attribute -> **multiple item selection (MIS)**

# Example - intersection



**Task:** We have two sequences and would like to list all **items** that **can be found in both**.

**Specification:**

In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $cis \in S[1..n]$ ,  $lrr \in S[1..m]$

Out:  $cnt \in \mathbb{N}$ ,  $both \in S[1..cnt]$

Pre: -

Post:

$(cnt, both) = \mathbf{MIS}(i=1..n, \mathbf{isin}(cis[i], lrr), cis[i])$

The isin function contains the „subtask”

we collect the values

# Example - intersection



**Task:** We have two sequences and would like to list all **items** that **can be found in both**.

try to decide whether a  
sequence (array, list)  
contains an item → decision

**Specification (isin):**

In:  $m \in \mathbb{N}$ ,  $lrr \in S[1..m]$ , **what**  $\in S$

Out:  $exists \in L$

Pre: -

Post:  $exists = \text{EXISTS}(j=1..m, \text{what}=lrr[j])$

# Example - intersection



**Task:** We have two sequences and would like to list all **items** that **can be found in both**.

**Specification:**

In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $cis \in S[1..n]$ ,  $lrr \in S[1..m]$

Out:  $cnt \in \mathbb{N}$ ,  $both \in S[1..cnt]$

Fn:  $isin: S \times S[] \rightarrow L$ ,

$isin(what, lrr) = \text{EXISTS}(j=1..m, what=lrr[j])$

Pre: -

Post:

$(cnt, both) = \text{MIS}(i=1..n, isin(cis[i], lrr), cis[i])$

*m is global*

# Example - intersection



**Task:** We have two sequences and would like to list all items that can be found in both.

## Specification:

In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $cis \in S[1..n]$ ,  $lrr \in S[1..m]$

Out:  $cnt \in \mathbb{N}$ ,  $both \in S[1..cnt]$

Fn:  $isin: S \times S[] \rightarrow L$ ,

$isin(what, lrr) = \text{EXISTS}(j=1..length(lrr), what=lrr[j])$

Pre: -

Post:  $(cnt, both) = \text{MIS}(i=1..n, isin(cis[i], lrr), cis[i])$

we can „get“ the  
length of a sequence  
(array)

# Example - intersection



## MIS pattern:

In:  $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out:  $\text{cnt} \in \mathbb{N}, y \in H[1..\text{cnt}]$

Pre: -

Post:  $(\text{cnt}, y) = \text{MIS}(i=b..e, A(i), f(i))$

## decision pattern:

In:  $b \in \mathbb{Z}, e \in \mathbb{Z}$

Out:  $\text{exists} \in \mathbb{L}$

Pre: -

Post:  $\text{exists} = \text{EXISTS}(i=b..e, A(i))$

## Specific task:

In:  $n \in \mathbb{N}, m \in \mathbb{N}, \text{cis} \in S[1..n], \text{lrr} \in S[1..m]$

Out:  $\text{cnt} \in \mathbb{N}, \text{both} \in S[1..\text{cnt}]$

Fn:  $\text{isin}: S \times S[] \rightarrow \mathbb{L}$ ,

$\text{isin}(\text{what}, \text{lrr}) =$

**$\text{EXISTS}(j=1..\text{length}(\text{lrr}), \text{what}=\text{lrr}[j])$**

Pre: -

Post:  $(\text{cnt}, \text{both}) =$

**$\text{MIS}(i=1..n, \text{isin}(\text{cis}[i], \text{lrr}), \text{cis}[i])$**

### MIS

$\text{cnt}, y \sim \text{cnt}, \text{both}$

$b..e \sim 1..n$

$A(i) \sim \text{isin}(\text{cis}[i], \text{lrr})$

### decision

$i \sim j$

$b..e \sim 1..m \text{ (length(lrr))}$

$A(i) \sim \text{what}=\text{lrr}[j]$

# Example - intersection



2 separate tasks.

## Patterns

|                                                                                |   |
|--------------------------------------------------------------------------------|---|
| <code>cnt:=0</code>                                                            |   |
| <code>i=b..e</code>                                                            |   |
| <div style="display: inline-block; width: 40%; text-align: center;">A(i)</div> |   |
| T                                                                              | F |
| <code>cnt:=cnt+1</code>                                                        | - |
| <code>y[cnt]:=f(i)</code>                                                      |   |

|                                                                               |
|-------------------------------------------------------------------------------|
| <code>i:=b</code>                                                             |
| <code>i&lt;=e and not A(i)</code>                                             |
| <div style="border: 1px solid black; padding: 2px;"><code>i:=i+1</code></div> |
| <code>exists:=(i&lt;=e)</code>                                                |

## Algorithm

|                                                                                            |   |
|--------------------------------------------------------------------------------------------|---|
| <code>cnt:=0</code>                                                                        |   |
| <code>i=1..n</code>                                                                        |   |
| <div style="display: inline-block; width: 40%; text-align: center;">isin(cis[i],lrr)</div> |   |
| T                                                                                          | F |
| <code>cnt:=cnt+1</code>                                                                    | - |
| <code>both[cnt]:=cis[i]</code>                                                             |   |

|                                                                               |
|-------------------------------------------------------------------------------|
| <code>j:=1</code>                                                             |
| <code>j&lt;=m and not (what=lrr[j])</code>                                    |
| <div style="border: 1px solid black; padding: 2px;"><code>j:=j+1</code></div> |
| <code>exists:=j&lt;=m</code>                                                  |

m=length(lrr)

## MIS

`cnt, y` ~ `cnt, both`  
`b..e` ~ `1..n`  
`A(i)` ~ `isin(cis[i],lrr)`

## decision

`i` ~ `j`  
`b..e` ~ `1..m (length(lrr))`  
`A(i)` ~ `what=lrr[j]`

# Example - intersection



## Algorithm

connecting:

1. **calling a function:** the nested algorithm will be a function and the „outer” algorithm call it
2. **integration:** copy the nested algorithm

|                                |                               |
|--------------------------------|-------------------------------|
| <code>cnt:=0</code>            |                               |
| <code>i=1..n</code>            |                               |
| <code>T</code>                 | <code>isin(cis[i],lrr)</code> |
| <code>cnt:=cnt+1</code>        | <code>F</code>                |
| <code>both[cnt]:=cis[i]</code> | <code>-</code>                |

|                                            |
|--------------------------------------------|
| <code>j:=1</code>                          |
| <code>j&lt;=m and not (what=lrr[j])</code> |
| <code>j:=j+1</code>                        |
| <code>exists:=j&lt;=m</code>               |

# Example - intersection



## Algorithm

connecting:

1. **calling a function:** the nested algorithm will be a function and the „outer” algorithm call it
2. **integration:** copy the nested algorithm

FOLLOW THIS!

```
cnt:=0
```

```
i=1..n
```

```
isin(cis[i],lrr)
```

T

F

```
cnt:=cnt+1
```

-

```
both[cnt]:=cis[i]
```

```
isin(what:String,where:String[]):Boolean
```

```
j:=1
```

```
j<=length(where) and not (what=where[j])
```

```
j:=j+1
```

```
isin:=(j<=length(where))
```

# Example - intersection



## Algorithm

### connecting:

1. **calling a function:** the nested algorithm will be a function and the „outer” algorithm call it
2. **integration:** copy the nested algorithm

|                                                                                               |   |                     |  |   |   |
|-----------------------------------------------------------------------------------------------|---|---------------------|--|---|---|
| <code>cnt:=0</code>                                                                           |   |                     |  |   |   |
| <code>i=1..n</code>                                                                           |   |                     |  |   |   |
| <code>exists:=isin(cis[i],lrr)</code>                                                         |   |                     |  |   |   |
| <table><tr><td colspan="2"><code>exists</code></td></tr><tr><td>T</td><td>F</td></tr></table> |   | <code>exists</code> |  | T | F |
| <code>exists</code>                                                                           |   |                     |  |   |   |
| T                                                                                             | F |                     |  |   |   |
| <code>cnt:=cnt+1</code>                                                                       | - |                     |  |   |   |
| <code>both[cnt]:=cis[i]</code>                                                                |   |                     |  |   |   |

|                                            |
|--------------------------------------------|
| <code>j:=1</code>                          |
| <code>j&lt;=m and not (what=lrr[j])</code> |
| <code>j:=j+1</code>                        |
| <code>exists:=j&lt;=m</code>               |

# Example - intersection



## Algorithm

### connecting:

1. **calling a function:** the nested algorithm will be a function and the „outer” algorithm call it
2. **integration:** copy the nested algorithm

|                                              |   |
|----------------------------------------------|---|
| <code>cnt:=0</code>                          |   |
| <code>i=1..n</code>                          |   |
| <code>j:=1</code>                            |   |
| <code>j&lt;=m and not (cis[i]=lrr[j])</code> |   |
| <code>j:=j+1</code>                          |   |
| <code>exists:=j&lt;=m</code>                 |   |
| <div>exists</div>                            |   |
| T                                            | F |
| <code>cnt:=cnt+1</code>                      | - |
| <code>both[cnt]:=cis[i]</code>               |   |

# Example – intersection, the code



```
static void Main(string[] args) {  
    string[] cis; string[] rrl;  
    List<string> both1 = new List<string>();  
  
    readIn(out cis, "Chair of the Indefinite Studies");  
    readIn(out rrl, "Lecturer in Recent Runes");  
    both1 = mis_both(cis,rrl);  
    writeOut(both1);  
}
```

# Example – intersection, the code



```
static void readIn(out string[] x, string text) {  
    int n;  
    Console.Error.Write("n of {0}= ", text);  
    int.TryParse(Console.ReadLine(), out n);  
    x = new string[n];  
    for (int i = 0; i < n; i++) {  
        Console.Error.Write("{0}. place = ", i + 1);  
        x[i] = Console.ReadLine();  
    }  
}  
  
static void writeOut(List<string> x) {  
    for (int i = 0; i < x.Count; i++) {  
        Console.Error.Write("{0}. place; ", x[i]);  
    }  
}
```

call 2 times:  
text give extra  
information

# Example – intersection, the code

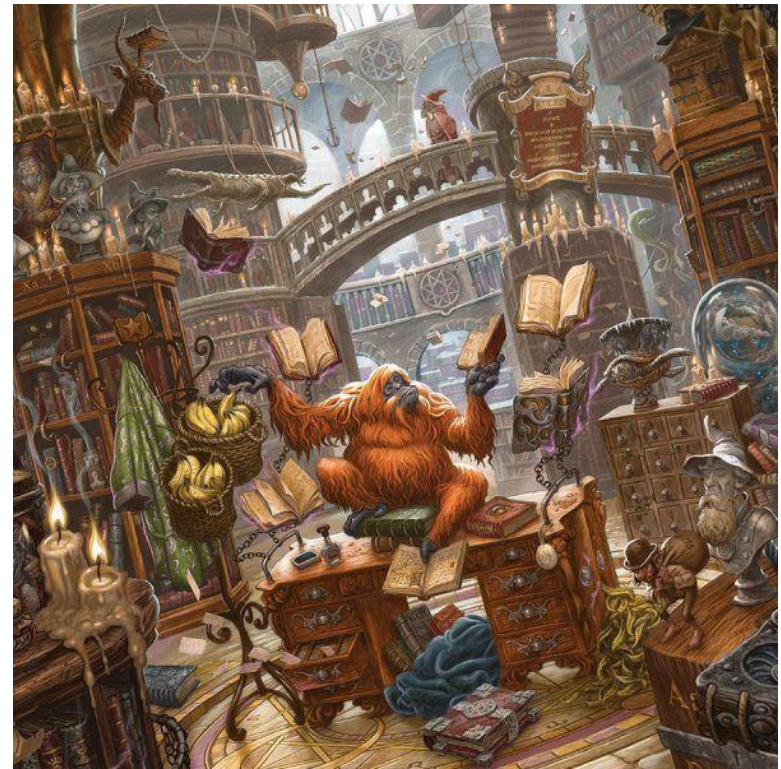
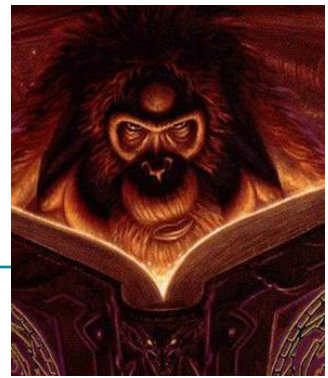


```
static List<string> mis_both(string[] x1, string[] x2) {  
    List<string> st = new List<string>();  
    for (int i= 0; i < x1.Length; i++) {  
        if (isIn(x1[i],x2)) {  
            st.Add(x1[i]);  
        }  
    }  
    return st;  
}  
  
static bool isIn(string what, string[] where) {  
    int j = 0;  
    while (j < where.Length &&! where[j].Equals(what)){  
        j++;  
    }  
    return (j < where.Length);  
}
```

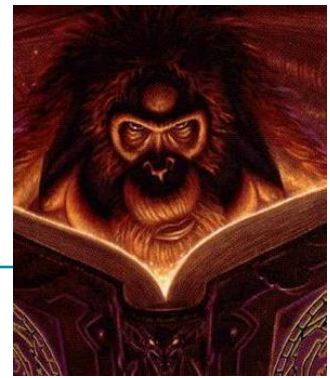
what<>where[j]

# Example – excellent

At the Unseen University, at the end of the year, we know the grade of  $n$  wizards in  $m$  subjects in a table. The Librarian would like to give the excellent students a banana. How many bananas should he buy?



# Example – excellent



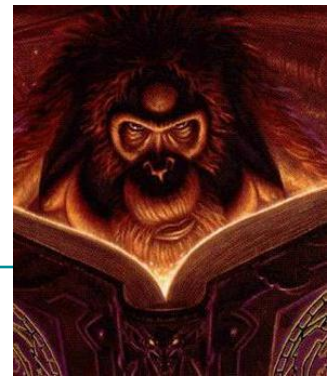
**Task:** how many rows contains only 5's in a matrix of integers?

|     | 1 | 2 | 3 | m=4 |
|-----|---|---|---|-----|
| 1   | 5 | 5 | 4 | 3   |
| 2   | 5 | 5 | 5 | 5   |
| 3   | 5 | 3 | 2 | 4   |
| 4   | 5 | 5 | 5 | 5   |
| n=5 | 5 | 3 | 4 | 5   |

Idea:

- how many rows do we have ...
- the answer is a number
- based on the **counting pattern**

# Example – excellent



**Task:** how many rows contains only 5's in a matrix of integers?

## Specification:

In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $\text{grades} \in \mathbb{N}[1..n, 1..m]$

Out:  $\text{cnt} \in \mathbb{N}$

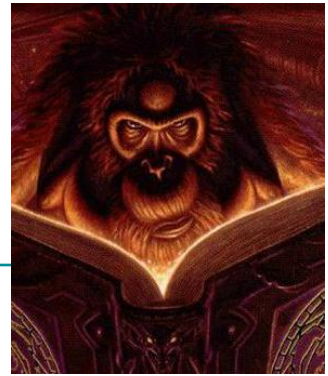
Pre:  $\forall r \in [1..n] : (\forall c \in [1..m] : (1 \leq \text{grades}[r, c] \leq 5))$

Post:  $\text{cnt} = \text{COUNT}(\text{row} = 1..n, \text{onlyfive}(\text{row}))$

The onlyfive function contains the „subtask”

|     | 1 | 2 | 3 | m=4 |
|-----|---|---|---|-----|
| 1   | 5 | 5 | 4 | 3   |
| 2   | 5 | 5 | 5 | 5   |
| 3   | 5 | 3 | 2 | 4   |
| 4   | 5 | 5 | 5 | 5   |
| n=5 | 5 | 3 | 4 | 5   |

# Example – excellent



**Task:** how many rows contains only 5's in a matrix of integers?

row that contains only 5's:

- focus on one row ...
- the answer is a true/false
- based on the **decision(all) pattern**

**Specification (onlyfive):**

In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $\text{grades} \in \mathbb{N}[1..n, 1..m]$ ,  $\text{row} \in \mathbb{N}$

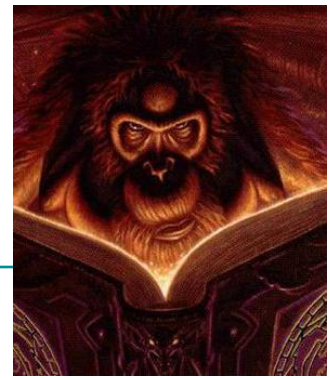
Out:  $\text{all} \in \mathbb{L}$

Pre:  $\forall r \in [1..n] : (\forall c \in [1..m] : (1 \leq \text{grades}[r, c] \leq 5))$

Post:  $(\text{all} = \text{onlyfive}(\text{row})) =$   
 $\text{all} = \text{ALL}(\text{col} = 1..m, \text{grades}[\text{row}, \text{col}] = 5)$

|     | 1 | 2 | 3 | m=4 |
|-----|---|---|---|-----|
| 1   | 5 | 5 | 4 | 3   |
| 2   | 5 | 5 | 5 | 5   |
| 3   | 5 | 3 | 2 | 4   |
| 4   | 5 | 5 | 5 | 5   |
| n=5 | 5 | 3 | 4 | 5   |

# Example – excellent



**Task:** how many rows contains only 5's in a matrix of integers?

row that contains only 5's:

- focus on one row ...
- the answer is a true/false
- based on the **decision(all) pattern**

**Specification (onlyfive):**

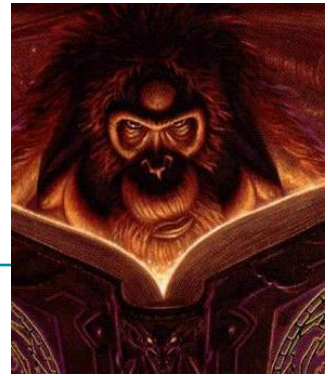
In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $\text{grades} \in \mathbb{N}[1..n, 1..m]$ ,  $\text{row} \in \mathbb{N}$

Out: `onlyfive(row)`  $\in \mathbb{L}$

Pre:  $\forall r \in [1..n] : (\forall c \in [1..m] : (1 \leq \text{grades}[r, c] \leq 5))$

Post: `onlyfive(row) = ALL(col = 1..m, grades[row, col] = 5)`

# Example – excellent



**Task:** how many rows contains only 5's in a matrix integers?

## Specification:

In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $\text{grades} \in \mathbb{N}[1..n, 1..m]$

Out:  $\text{cnt} \in \mathbb{N}$

Fn:  $\text{onlyfive}: \mathbb{N} \rightarrow \mathbb{L}$ ,

$\text{onlyfive}(\text{row}) = \text{all} \in \mathbb{L}:$

$\text{ALL}(\text{col} = 1..m, \text{grades}[\text{row}, \text{col}] = 5))$

Pre:  $\forall r \in [1..n]: (\forall c \in [1..m]: (1 \leq \text{grades}[r, c] \leq 5))$

Post:  $\text{cnt} = \text{COUNT}(\text{row} = 1..n, \text{onlyfive}(\text{row}))$

grades, n, m are global

„all” for which the part after : is true

|     | 1 | 2 | 3 | m=4 |
|-----|---|---|---|-----|
| 1   | 5 | 5 | 4 | 3   |
| 2   | 5 | 5 | 5 | 5   |
| 3   | 5 | 3 | 2 | 4   |
| 4   | 5 | 5 | 5 | 5   |
| n=5 | 5 | 3 | 4 | 5   |



ELTE

FACULTY OF  
INFORMATICS

PROGRAMMING – Zsuzsa Pluhár

# Example – excellent



## counting pattern:

In:  $b \in \mathbb{Z}$ ,  $e \in \mathbb{Z}$

Out:  $\text{cnt} \in \mathbb{N}$

Pre: -

Post:  $\text{cnt} = \text{COUNT}(i=b..e, A(i))$

## decision all pattern:

In:  $b \in \mathbb{Z}$ ,  $e \in \mathbb{Z}$

Out:  $\text{all} \in \mathbb{L}$

Pre: -

Post:  $\text{all} = \text{ALL}(i=b..e, A(i))$

## specific task:

In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $\text{grades} \in \mathbb{N}[1..n, 1..m]$

Out:  $\text{cnt} \in \mathbb{N}$

Fn:  $\text{onlyfive}: \mathbb{N} \rightarrow \mathbb{L}$ ,

$\text{onlyfive}(\text{row}) = \text{all} \in \mathbb{L}$ :

$\text{ALL}(\text{col}=1..m, \text{grades}[\text{row}, \text{col}]=5))$

Pre: ...

Post:  $\text{cnt} = \text{COUNT}(\text{row}=1..n, \text{onlyfive}(\text{row}))$

### counting

$i \sim \text{row}$

$b..e \sim 1..n$

$A(i) \sim \text{onlyfive}(\text{row})$

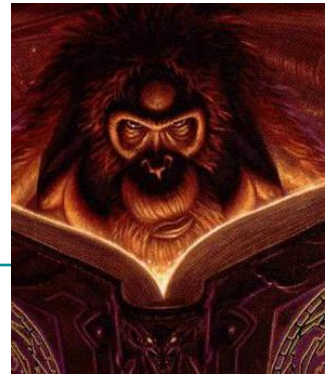
### decision all

$i \sim \text{col}$

$b..e \sim 1..m$

$A(i) \sim \text{grades}[\text{row}, \text{col}]=5$

# Example – excellent



**Task:** how many rows contains only 5's in a matrix of integers?

## Specification:

In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $\text{grades} \in \mathbb{N}[1..n, 1..m]$

Out:  $\text{cnt} \in \mathbb{N}$

Fn:  $\text{onlyfive}: \mathbb{N} \rightarrow \mathbb{L}$ ,

$\text{onlyfive}(\text{row}) = \text{all} \in \mathbb{L}:$

$\text{ALL}(\text{col} = 1..m, \text{grades}[\text{row}, \text{col}] = 5)$

Pre:  $\forall r \in [1..n]: (\forall c \in [1..m]: (1 \leq \text{grades}[r, c] \leq 5))$

Post:  $\text{cnt} = \text{COUNT}(\text{row} = 1..n, \text{onlyfive}(\text{row}))$

This redundant information will be omitted in the following. Assume that it specifies the output of the pattern! It will not appear in the reduction.

### counting

$i \sim \text{row}$

$b..e \sim 1..n$

$A(i) \sim \text{onlyfive}(\text{row})$

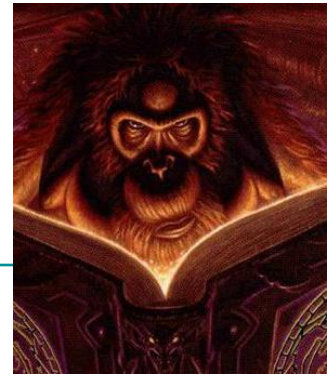
### decision all

$i \sim \text{col}$

$b..e \sim 1..m$

$A(i) \sim \text{grades}[\text{row}, \text{col}] = 5$

# Example – excellent



**Task:** how many rows contains only 5's in a matrix of integers?

## Specification:

In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $\text{grades} \in \mathbb{N}[1..n, 1..m]$

Out:  $\text{cnt} \in \mathbb{N}$

Fn: onlyfive:  $\mathbb{N} \rightarrow \mathbb{L}$ ,

onlyfive(row) =  $\text{ALL}(\text{col} = 1..m, \text{grades}[\text{row}, \text{col}] = 5)$

Pre:  $\forall r \in [1..n]: (\forall c \in [1..m]: (1 \leq \text{grades}[r, c] \leq 5))$

Post:  $\text{cnt} = \text{COUNT}(\text{row} = 1..n, \text{onlyfive}(\text{row}))$

### counting

$i \sim \text{row}$

$b..e \sim 1..n$

$A(i) \sim \text{onlyfive}(\text{row})$

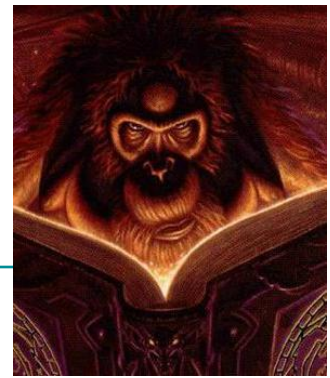
### decision all

$i \sim \text{col}$

$b..e \sim 1..m$

$A(i) \sim \text{grades}[\text{row}, \text{col}] = 5$

# Example – excellent



**Comment:** the patterns can be combined

**Specification:**

In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $\text{grades} \in \mathbb{N}[1..n, 1..m]$

Out:  $\text{cnt} \in \mathbb{N}$

Fn:  $\text{onlyfive}: \mathbb{N} \rightarrow \mathbb{L}$ ,

$\text{onlyfive}(\text{row}) = \text{ALL}(\text{col} = 1..m, \text{grades}[\text{row}, \text{col}] = 5)$

Pre:  $\forall r \in [1..n]: (\forall c \in [1..m]: (1 \leq \text{grades}[r, c] \leq 5))$

Post:  $\text{cnt} = \text{COUNT}(\text{row} = 1..n, \text{onlyfive}(\text{row}))$



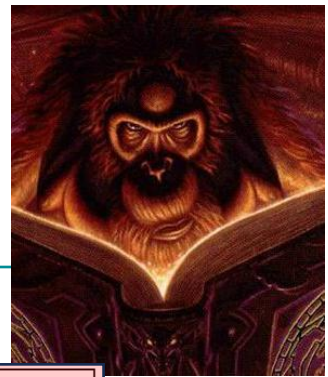
In:  $n \in \mathbb{N}$ ,  $m \in \mathbb{N}$ ,  $\text{grades} \in \mathbb{N}[1..n, 1..m]$

Out:  $\text{cnt} \in \mathbb{N}$

Pre:  $\forall r \in [1..n]: (\forall c \in [1..m]: (1 \leq \text{grades}[r, c] \leq 5))$

Post:  $\text{cnt} = \text{COUNT}(\text{row} = 1..n, \text{ALL}(\text{col} = 1..m, \text{grades}[\text{row}, \text{col}] = 5))$

# Example – excellent



## Patterns

|                         |      |
|-------------------------|------|
| <code>cnt:=0</code>     |      |
| <code>i=b..e</code>     |      |
| T                       | A(i) |
| <code>cnt:=cnt+1</code> | -    |

|                                                                                    |                     |
|------------------------------------------------------------------------------------|---------------------|
| <code>i:=b</code>                                                                  |                     |
| <code>i&lt;=e and A(i)</code> <table><tr><td><code>i:=i+1</code></td></tr></table> | <code>i:=i+1</code> |
| <code>i:=i+1</code>                                                                |                     |
| <code>all:=(i&gt;e)</code>                                                         |                     |

## Algorithm

|                         |               |
|-------------------------|---------------|
| <code>cnt:=0</code>     |               |
| <code>row=1..n</code>   |               |
| T                       | onlyfive(row) |
| <code>cnt:=cnt+1</code> | -             |

2 separate tasks.

|                                              |                         |
|----------------------------------------------|-------------------------|
| <code>col:=1</code>                          |                         |
| <code>col&lt;=m and grades[col,row]=5</code> | <code>col:=col+1</code> |
| <code>all:=(col&gt;m)</code>                 |                         |

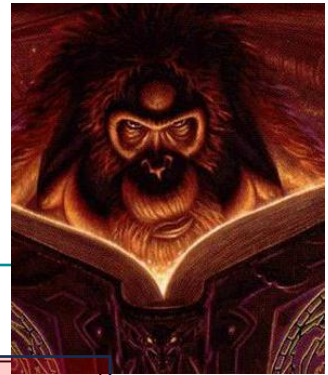
### counting

`i` ~ `row`  
`b..e` ~ `1..n`  
`A(i)` ~ `onlyfive(row)`

### decision all

`i` ~ `col`  
`b..e` ~ `1..m`  
`A(i)` ~ `grades[row,col]=5`

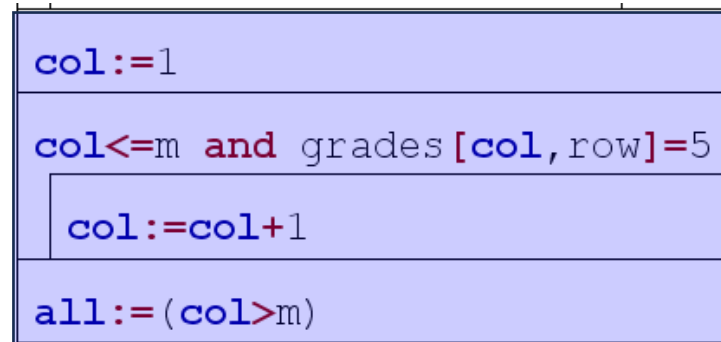
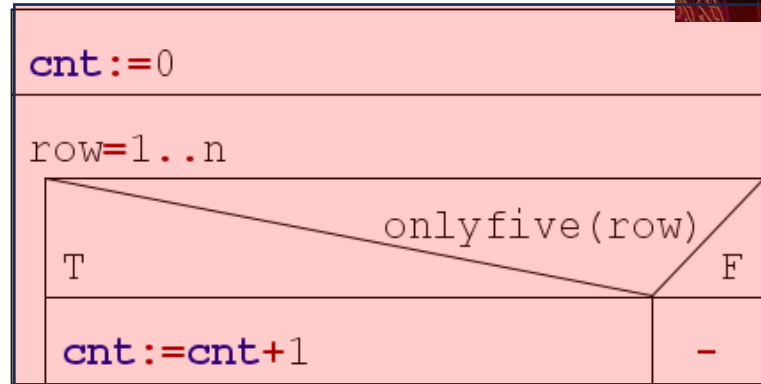
# Example – excellent



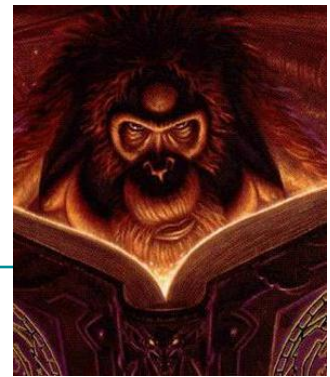
connecting:

1. **calling a function:** the nested algorithm will be a function and the „outer” algorithm call it
2. **integration:** copy the nested algorithm

## Algorithm



# Example – excellent



connecting:

1. **calling a function:** the nested algorithm will be a function and the „outer” algorithm call it
2. **integration:** copy the nested algorithm

FOLLOW THIS!

## Algorithm

|                         |                          |
|-------------------------|--------------------------|
| <code>cnt:=0</code>     |                          |
| <code>i=1..n</code>     |                          |
| <code>T</code>          | <code>onlyfive(i)</code> |
| <code>cnt:=cnt+1</code> | <code>-</code>           |
| <code>F</code>          |                          |

```
onlyfives (row:Integer) : Boolean

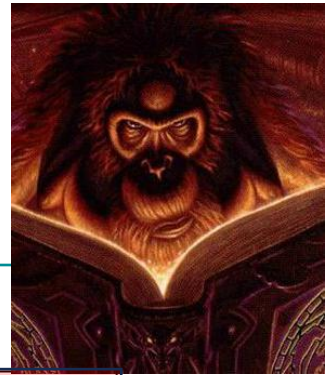
col:=1

col<=m and grades[col,row]=5
  col:=col+1

all:=(col>m)

onlyfives:=all
```

# Example – excellent



## Algorithm

### connecting:

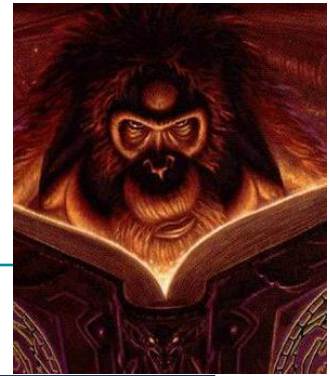
1. calling a function: the nested algorithm will be a function and the „outer” algorithm call it
2. integration: copy the nested algorithm

1st step: helper variable

|                                 |   |
|---------------------------------|---|
| <code>cnt:=0</code>             |   |
| <code>row=1..n</code>           |   |
| <code>all:=onlyfive(row)</code> |   |
| T                               | F |
| <code>cnt:=cnt+1</code>         | - |

|                                              |
|----------------------------------------------|
| <code>col:=1</code>                          |
| <code>col&lt;=m and grades[col,row]=5</code> |
| <code>col:=col+1</code>                      |
| <code>all:=(col&gt;m)</code>                 |

# Example – excellent



## Algorithm

### connecting:

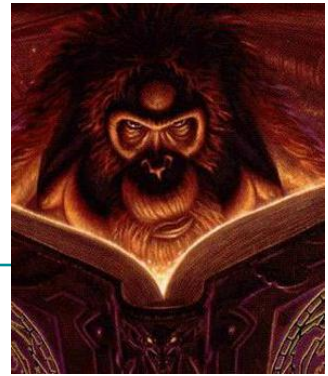
1. calling a function: the nested algorithm will be a function and the „outer” algorithm call it
2. integration: copy the nested algorithm

1st step: integration

|                                              |   |
|----------------------------------------------|---|
| <code>cnt:=0</code>                          |   |
| <code>row=1..n</code>                        |   |
| <code>col:=1</code>                          |   |
| <code>col&lt;=m and grades[col,row]=5</code> |   |
| <code>col:=col+1</code>                      |   |
| <code>all:=(col&gt;m)</code>                 |   |
| T                                            | F |
| <code>cnt:=cnt+1</code>                      | - |

# Example – excellent, the code

---



# Conclusions

---

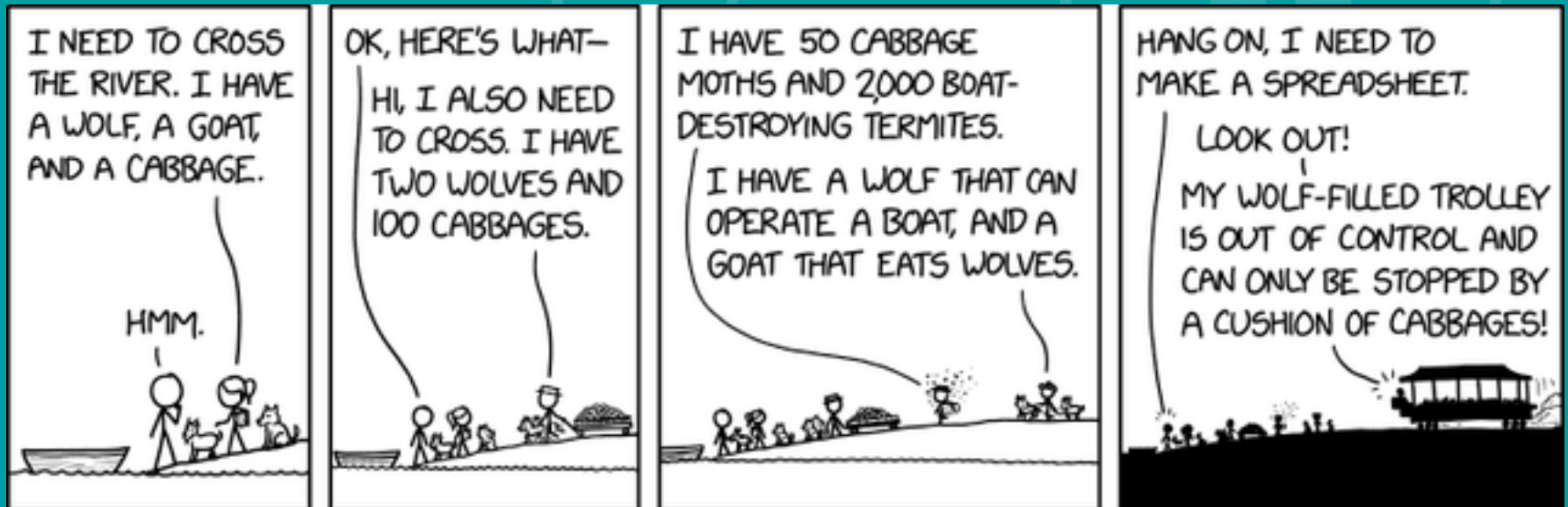
1. In a matrix, for each dimension can be (at least) one pattern
2. Try to break it down into subproblems (from the top to bottom)
3. Implement as function (algorithm, code)





# ELTE

FACULTY OF  
INFORMATICS



Thank you for your attention!